



HABIB UNIVERSITY

Data Structures & Algorithms

CS/CE 102/171 Spring 2025

Instructor: Maria Samad

Quiz # 2 Solution

QUESTION 1: ALGORITHM DESIGN USING CIRCULAR QUEUES (10 marks)

- **Log Management in a Circular Queue:**
 - The user inputs an expiration time in minutes, which should be an integer of not more than 59. If the expiration time is not in the valid range, continue taking input until right value is received.
 - Design an algorithm to design and manage a circular queue to manage system log entries, as explained below.
 - Circular Queue is of size 'n', which is initially empty. Assume, both circular queue and n are correct, so you may use them without error checking and taking inputs. However, you must declare/define them in your algorithm.
 - Each log entry is defined as a tuple consisting of the timestamp at which it occurs, and a string message, which is the log message. See example below to know the exact format.
 - The log entries are going to expire after the input expiration time in the first step is over, meaning that any log older than the expiration time should automatically be removed from the circular queue.
 - This means that before enqueueing log entries, the previous log entries must be checked, and if the time difference between their timestamps and current time is beyond the expiration time, it should be dequeued.
 - Once the expired log entries have been removed, only then the current log entry should be added to the circular queue, according to the behavior of circular queues.
 - While adding, if the queue is already full, it should first remove the oldest log before adding a new one.
 - For example:
 - Circular Queue size is 5, and expiration time is 10 minutes
 - Initial Circular Queue of size 5 = [None, None, None, None, None]
 - Front pointer = Rear pointer = None
 - Summary of Operations, as the log stamp appears is as follows:
 1. Enqueue (1, "Log 1"): Queue = [(1, "Log 1"), None, None, None, None]
 2. Enqueue (3, "Log 2"): Queue = [(1, "Log 1"), (3, "Log 2"), None, None, None]
 3. Enqueue (5, "Log 3"): Queue = [(1, "Log 1"), (3, "Log 2"), (5, "Log 3"), None, None]
 4. Enqueue (15, "Log 4") (remove expired logs): Queue = [(5, "Log 3"), (15, "Log 4"), None, None, None]
 5. Enqueue (20, "Log 5") (remove expired logs): Queue = [(15, "Log 4"), (20, "Log 5"), None, None, None]
 6. Enqueue (21, "Log 6"): Queue = [(15, "Log 4"), (20, "Log 5"), (21, "Log 6"), None, None]
 7. Enqueue (22, "Log 7"): Queue = [(15, "Log 4"), (20, "Log 5"), (21, "Log 6"), (22, "Log 7"), None]
 8. Enqueue (23, "Log 8"): Queue = [(15, "Log 4"), (20, "Log 5"), (21, "Log 6"), (22, "Log 7"), (23, "Log 8")]
 9. Enqueue (24, "Log 9") (queue is full so remove the front element, which in this situation at the moment is "Log 4" timestamp, to make new room and then add (24, "Log 9") to CQ as following: Queue = [(20, "Log 5"), (21, "Log 6"), (22, "Log 7"), (23, "Log 8"), (24, "Log 9")]
 10. Enqueue (-1, "EXIT"): Indication to stop the program
- **REQUIREMENTS:**
 - This is a question on CIRCULAR QUEUES, so please stick to ONLY Circular Queues for the algorithm. The enqueue and dequeue functions do NOT handle the CQ pointers, so you should handle them in your algorithm.
 - Do not **HARD CODE** your design for the given example. The algorithm should work for any valid sized CQs.
 - Declare all the data structures and variables being used in this algorithm, including any inputs, if any.
 - The program stops when a log entry with timestamp -1 and string message, "EXIT" occurs
 - The algorithm should be able to handle new log additions and automatically discard expired/old entries efficiently, i.e. in constant time, as per the behavior of circular queues
 - **DO NOT USE ANY OTHER DATA STRUCTURES INCLUDING ListADT/Python lists apart from the ones permitted to use**
 - **REMEMBER:** Circular Queues are non-iterable and non-traversing data structures, so you CANNOT and MUST NOT do indexing within the data structure to access the elements.

SOLUTION:

- expiry_time – take integer input from the user between the range of 0 and 59 (both inclusive)
- If not in valid range, prompt the user to enter again, and continue until the right value is input
- n – an integer representing the size of Circular Queue
- CQ – Circular Queue of size n
- front – pointer to keep track of the front element, initially set to None
- rear – a pointer to keep track of the last element, initially set to None
- log_entry – a tuple with timestamp as integer key, and message as string value. There's no requirement mentioned for it, so you can assume any possible situation of taking this as an input to the program. It can be done by reading a file, or some array where there are values present, or even taking input from the user.

#Checking for exit condition

- if log_entry[0] == -1 and log_entry[1] == "EXIT", then end the program by doing as follows:
 - return

Otherwise update the CQ accordingly

- while log_entry[0] != -1 and log_entry[1] != "EXIT", then do the following:
 - log_time = CQ.front[0]
 - if log_entry[0] – log_time >= expiry_time, then do as follows:
 - if front == rear:
 - front = None
 - rear = None
 - Otherwise, do as the following:
 - front = (front + 1) % n
 - Otherwise, break out of the loop
- If CQ is full:
 - front = (front + 1) % n
- Else if CQ is empty:
 - front = 0
- Otherwise, update the pointers
 - rear = (rear + 1) % n
- CQ.enqueue(log_entry)

QUESTION 2: BINARY SEARCH OPERATIONS (10 marks)

A sorted list of student IDs is given as follows: **studIDs = [3, 7, 12, 18, 24, 32, 39, 45, 51, 62, 71]**

Answer the following questions based on the binary search process, by showing all the steps and/or calculations. Only giving final answers will **NOT** be given any credit.

1. List the sequence of indices that the binary search algorithm will examine while searching for student ID = 32.
2. Suppose the student mistakenly searches for roll number 60 instead. List the sequence of indices examined and explain how binary search determines that 60 is not in the array.
3. List the sequence of indices that the binary search algorithm will examine while searching for student ID = 3.
4. If a new roll number 34 is added to the list while maintaining sorted order, what is the effect on the binary search process when searching for 32, and then 39? Which indices would now be visited?
5. If 18 is deleted, how does the search path change for 24 and 12?
6. What is the maximum number of comparisons binary search makes in a sorted array of size 1000?
7. Given a sorted array of 19 elements, what will be the first middle index chosen by binary search? What if the array had 20 elements instead?

SOLUTION:

1. List the sequence of indices that the binary search algorithm will examine while searching for student ID = 32.
 - Find middle index to go to index 5, where this value is being searched at the moment and it is indeed 32.
Therefore, the sequence of indices is: **5 → SUCCESS**
2. Suppose the student mistakenly searches for roll number 60 instead. List the sequence of indices examined and explain how binary search determines that 60 is not in the array.
 - Go to middle index first, i.e. index 5. The element at index 5 = 32 and $60 > 32$, so search the right side of the array, i.e. from index 6 to index 10
 - Then go to the new middle index, i.e. index 8. The element at index 8 = 51 and $60 > 51$, so search the right side of the array, i.e. from index 9 to index 10
 - Then go to the new middle index, i.e. index 9. The element at index 9 = 62 and $60 < 62$, so we are supposed to search for the left side of the array. However, the ending condition is met so we stop and deduce 60 is not present in the array.
Therefore, the sequence of indices is: **5 → 8 → 9 → FAILURE**
3. List the sequence of indices that the binary search algorithm will examine while searching for student ID = 3.
 - Go to middle index first, i.e. index 5. The element at index 5 = 32 and $3 < 32$, so search the left side of the array, i.e. from index 0 to index 4
 - Then go to the new middle index, i.e. index 2. The element at index 2 = 12 and $3 < 12$, so search the left side of the array, i.e. from index 0 to index 1
 - Then go to the new middle index, i.e. index 0. The element at index 0 = 3, which is the value that's being searched at the moment.
 - Therefore, the sequence of indices is: **5 → 2 → 0 → SUCCESS**
4. If a new roll number 34 is added to the list while maintaining sorted order, what is the effect on the binary search process when searching for 32, and then 39? Which indices would now be visited?
 - When 34 is added, the updated array will be as follows:
 - **studIDs = [3, 7, 12, 18, 24, 32, 34, 39, 45, 51, 62, 71]**
 - Now search for 32:
 - Find middle index to go to index 5, where this value is being searched at the moment and it is indeed 32.
 - Therefore, the sequence of indices is: **5 → SUCCESS**
 - No change as 32 is still the middle element
 - Now search for 39:
 - Go to middle index first, i.e. index 5. The element at index 5 = 32 and $39 > 32$, so search the right side of the array, i.e. from index 6 to index 11.
 - Then go to the new middle index, i.e. index 8. The element at index 8 = 45 and $39 < 45$, so search the left side of the array, i.e. from index 6 to index 7
 - Then go to the new middle index, i.e. index 6. The element at index 6 = 34 and $39 > 34$, so search the right side of the array, i.e. index 7
 - Then go to the new middle index, i.e. index 7. The element at index 7 = 39, which is the value that's being searched at the moment
 - Therefore, the sequence of indices is: **5 → 8 → 6 → 7 → SUCCESS**
 - Similar process but for different set of indices instead
5. If 18 is deleted, how does the search path change for 24 and 12?
 - When 18 is deleted, the updated array will be as follows:
 - **studIDs = [3, 7, 12, 24, 32, 34, 39, 45, 51, 62, 71]**
 - Now search for 24:
 - Go to middle index first, i.e. index 5. The element at index 5 = 34 and $24 < 34$, so search the left side of the array, i.e. from index 0 to index 4.
 - Then go to the new middle index, i.e. index 2. The element at index 2 = 12 and $24 > 12$, so search the right side of the array, i.e. from index 3 to index 4
 - Then go to the new middle index, i.e. index 3. The element at index 3 = 24, which is the value that's being searched at the moment
 - Therefore, the sequence of indices is: **5 → 8 → 6 → SUCCESS**

- Similar process but for different set of indices instead
- Now search for 12:
 - Go to middle index first, i.e. index 5. The element at index 5 = 34 and $12 < 32$, so search the left side of the array, i.e. from index 0 to index 4.
 - Then go to the new middle index, i.e. index 2. The element at index 2 = 12, which is the value that's being searched at the moment
 - Therefore, the sequence of indices is: **5 → 2 → SUCCESS**
 - Similar process but for different set of indices instead
- 6. What is the maximum number of comparisons binary search makes in a sorted array of size 1000?
 - Search Time in Binary Search by default contains $O(\log n)$. If $n = 1000$, then number of comparisons will be: $\log_2(1000) = 9.966$
- 7. Given a sorted array of 19 elements, what will be the first middle index chosen by binary search? What if the array had 20 elements instead?
 - For 19 elements: middle = $(0 + 18)/2 = 9$ hence it will be: **9**
 - For 20 elements: middle = $(0 + 19)/2 = 9$ hence it will also be: **9**

QUESTION 3: BUBBLE SORT (10 marks)

Sort the given list in descending order using bubble sort algorithm.

- [43, 92, 31, 14, 27, 65, 79, 16, 58, 90, 2]

SHOW ALL INTERIM CALCULATIONS IN ORDER TO GET COMPLETE/FULL MARKS. An answer without the intermediate steps will directly result in 0 marks.

SOLUTION: On next pages

Date: Q3 Bubble Sort.

[43, 92, 31, 14, 27, 65, 79, 16, 58, 90, 2]

First Pass:

[43, 92, 31, 14, 27, 65, 79, 16, 58, 90, 2]
92 43 31 27 14 14 14 14 14 14 2
65 79 16 58 90

∴ [92, 43, 31, 27, 65, 79, 16, 58, 90, 14, 2]

Second Pass

[92, 43, 31, 27, 65, 79, 16, 58, 90, 14, 2]
92 43 31 27 27 27 27 58 16 16 2
65 79 90

∴ [92, 43, 31, 65, 79, 27, 58, 90, 16, 14, 2]

Third Pass

[92, 43, 31, 65, 79, 27, 58, 90, 16, 14, 2]
92 43 65 31 31 58 27 27 16 14 2
79 90

∴ [92, 43, 65, 79, 31, 58, 90, 27, 16, 14, 2]

Fourth Pass

[92, 43, 65, 79, 31, 58, 90, 27, 16, 14, 2]
92 65 43 43 58 31 31 27 16 14 2
79 90

∴ [92, 65, 79, 43, 58, 90, 31, 27, 16, 14, 2]

Date: _____

Fifth Pass

[92, 65, 79, 43, 58, 90, 31, 27, 16, 14, 2]
92 79 65 58 43 31 27 16 14 2
90

$\therefore$ [92, 79, 65, 58, 90, 43, 31, 27, 16, 14, 2]

Sixth Pass

[92, 79, 65, 58, 90, 43, 31, 27, 16, 14, 2]
92 79 65 90 58 43 31 27 16 14 2

$\therefore$ [92, 79, 65, 90, 58, 43, 31, 27, 16, 14, 2]

Seventh Pass

[92, 79, 65, 90, 58, 43, 31, 27, 16, 14, 2]
92 79 90 65 58 43 31 27 16 14 2

$\therefore$ [92, 79, 90, 65, 58, 43, 31, 27, 16, 14, 2]

Eighth Pass

[92, 79, 90, 65, 58, 43, 31, 27, 16, 14, 2]
92 90 79 65 58 43 31 27 16 14 2

$\therefore$ [92, 90, 79, 65, 58, 43, 31, 27, 16, 14, 2]

In the ninth pass, no swapping occurs & it's sorted now!

DALMATIAN